

IMAGE SYNTHESIS ON LOW COMPUTE MACHINES

A DISSERTATION SUBMITTED TO THE UNIVERSITY OF MANCHESTER
FOR THE DEGREE OF MASTER OF SCIENCE
IN THE FACULTY OF SCIENCE AND ENGINEERING

2024

Student id: 10890214

Department of Computer Science

Contents

List of Tables	5
List of Figures	6
Abstract	8
Acknowledgements	9
1 Introduction	10
1.1 Motivation	10
1.2 Aims and Objectives	10
1.3 Related Work	11
1.4 Existing Lightweight Models	12
1.4.1 Crayon (DALL-E Mini)	12
1.4.2 TinyGAN	13
1.4.3 Knowledge Distillation	13
1.5 Evaluation Strategy	13
2 Background	15
2.1 Image processing	15
2.2 Convolutions	15
2.3 Common Image Synthesis Architectures	16
2.3.1 Flow Models	16
2.3.2 Variational Auto Encoders (VAEs)	16
2.3.3 Diffusion Models	17
2.3.4 Generative Adversarial Networks (GANs)	17
2.3.5 Why GANs and Diffusion	17
2.4 GANs in Depth	18
2.4.1 The Generator	18
2.4.2 The Discriminator	20
2.4.3 Training Process	20
2.4.4 Diffusion Models in Depth	22

3	Methodology	24
3.1	Hardware	24
3.2	Latent Diffusion U-Net Implementation	24
3.2.1	Residual Blocks	24
3.2.2	Down Blocks	25
3.2.3	Up Blocks	25
3.2.4	Residual Connections	25
3.2.5	The Final Residual Block Design	27
3.2.6	Time Schedule	28
3.2.7	Diffusion Loss Function	30
3.3	GAN Implementation	30
3.3.1	Generator Implementation	30
3.3.2	Discriminator Implementation	31
3.4	Requirements	31
3.4.1	Dataset	31
3.4.2	Data Preprocessing	32
3.5	Training	32
3.6	Application Design	32
3.6.1	Functionality	33
3.6.2	User Interface	33
4	Evaluation	35
4.1	Model Evaluations	35
4.1.1	Inception Score (IS)	35
4.2	Model Performance Review	38
4.2.1	Quantitative Performance Review	39
4.2.2	Model Survey Results Analysis	39
4.3	Image Examples	44
4.3.1	Black and White General Adversarial Model Images	44
4.3.2	Coloured General Adversarial Images	45
4.3.3	Diffusion Model Images	46
4.4	Evaluation of the Application	46
4.5	Application Review	47
5	Conclusion	50
5.1	Achievements and Improvements	50
5.2	Future Work	51
5.3	Reflection and Conclusion	51

Word Count: 11,168

List of Tables

4.1	Model Performance (In bold are the models created for this paper)	39
-----	---	----

List of Figures

1.1	Images Generated by DALL-E 1 [20]	11
1.2	Images Generated by DALL-E 2 [2]	11
1.3	Images Generated by Midjourney [2]	12
1.4	Images Generated by Stable Diffusion [2]	12
1.5	Images Generated by Stable Craiyon [24]	12
1.6	BigGan images on top, TinyGan images on bottom [5]	13
2.1	Single Convolutional Operation [8]	16
2.2	Deconv Operation [36]	19
2.3	Generator Overview [19]	19
2.4	GAN Training Overview [11]	21
2.5	U-Net Overview [28]	22
2.6	Noise Operation Graph [14]	23
3.1	The Attention Mechanism [30]	26
3.2	U-Net Residual Connections within a U-Net [28]”	27
3.3	Encoder and Decoder Blocks described in Semantic Image Synthesis via Dif- fusion Models [32]	27
3.4	Implementation of residual block used for this project	28
3.5	Implementation of residual block used for this project	28
3.6	Final User Interface Design	34
4.1	Label Distribution [17]	36
4.2	Marginal Distribution [17]	37
4.3	KL-Divergence [17]	37
4.4	Feedback on Realism of Images	39
4.5	Feedback on Image Diversity	40
4.6	Feedback on Generation Speed	41
4.7	Feedback on Consistency of Image Quality	42
4.8	Set of BnW Images	44
4.9	Set of Coloured GAN Images	45

4.10 Set of Diffusion Model Images	46
4.11 Feedback on UI Learnability	47
4.12 Feedback on UI Navigation Efficiency	48
4.13 Feedback on UI Aesthetics	48
4.14 Feedback on UI Consistency	49

Abstract

Recent state-of-the-art models such as ChatGPT and Dall-e 2 are extremely large, and cannot be run on the average home computer. As AI in the form of deep neural networks permeates our society there is a growing need to ensure these models are inclusive and accessible to all, rather than a select few with the proper resources. In response, this paper explores the possibility of training small lightweight models from scratch using the techniques of latent diffusion and Generative adversarial networks (GANs) to produce high-fidelity images without the need for extensive computational resources. The final product will be an application that can generate images while running locally even without a dedicated GPU on the machine, this will demonstrate whether the latest techniques in the field can help to bridge this gap in accessibility. Through empirical evaluation and comparisons with existing methods, the paper aims to evaluate the feasibility and performance of these techniques on low-compute machines.

Acknowledgements

I would like to thank my supervisor Dr. Tingting Mu, for always being supportive and accommodating throughout the year, especially when progress with the project was slow and/or difficult. I would also like to thank my family and friends for their encouragement, and their ability to put up with me during particularly stressful exam seasons.

Chapter 1

Introduction

1.1 Motivation

The current landscape of AI image generation tools is shifting rapidly in the direction of larger and larger models which demand higher and higher compute capabilities. While these large models are able to produce high-fidelity images, they have led to the average user being unable to use these tools on local machines which do not have the necessary compute. This trend leads to issues concerning the democratisation of AI tools in general as it excludes a large portion of the population. While methods of knowledge distillation and other forms of model compression can perform well in reducing the computational performance needed for their use, they are still reliant on the initially very large models and their pre-training, and thus still run into the same issue. Even approaches specifically designed to reduce the size of such models, such as the one proposed by Cho et al. [6] were only able to reduce the size of Llama 7B down to 2.5GB which cannot feasibly be trained or used on a home device. The technique of diffusion models has been shown to perform well at producing high fidelity images with relatively low amounts of compute by iteratively denoising images [1], compared with techniques such as GANs [12], which arguably perform worse. For example, Nvidias styleGan 3 has 530 billion parameters whereas OpoenAIs Dalle2 contains only 3.5 billion. The discrepancy in performance and size of these approaches suggests that for a low-power device, new advances in the field of deep learning like diffusion models may be able to bridge the gap between large and small models. To test this I will produce both generative adversarial networks and diffusion models, comparing the two using quantitative and qualitative evaluation methods.

1.2 Aims and Objectives

- Investigate and explore possible approaches to training image-generation models
- Implement the most promising techniques and train using a large image dataset

- Create a clear usable application that allows the use of the produced models
- Use qualitative and quantitative methods to assess the relative performance of these models
- Compare these lightweight models with current state-of-the-art models

1.3 Related Work

The last two years have seen a major shift in the capabilities of models used to generate images due to the exponential take-off that has occurred. For example, DALL-E 1 was released in the January of 2021, and DALL-E 2 in April 2022, In this small period OpenAI was able to find significant performance gains, as can be seen in the figures below.



Figure 1.1: Images Generated by DALL-E 1 [20]

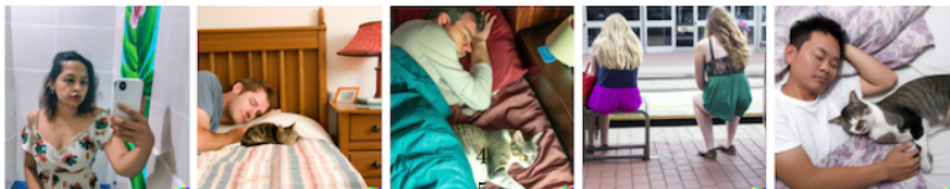


Figure 1.2: Images Generated by DALL-E 2 [2]

One of the major changes made during the development of DALL-E 2 is the architecture of the model. While DALL-E 1 is an implementation of a discrete variational autoencoder, DALL-E 2 utilises a latent diffusion architecture, a type of diffusion model that gradually refines an initially noisy image into something interpretable. DALL-E 2 is also much smaller than DALL-E 1, having 3.5 billion and 12 billion parameters respectively, suggesting the use of latent diffusion for a lightweight implementation of an AI image-generating model may be viable within the context of this project. It is also worth noting that this shift is not unique to OpenAI's DALL-E 2, with Midjourney and Stable diffusion all implementing variations on diffusion models with excellent results.

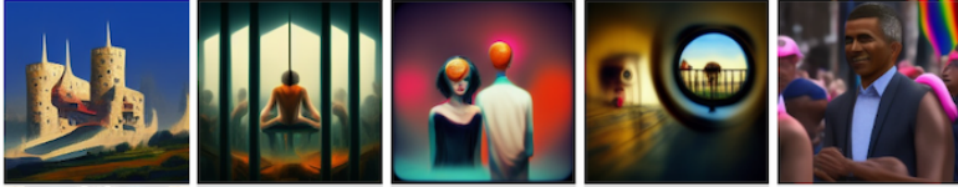


Figure 1.3: Images Generated by Midjourney [2]



Figure 1.4: Images Generated by Stable Diffusion [2]

1.4 Existing Lightweight Models

The main objective of this paper is to produce small models for image synthesis that can be run on machines without a dedicated GPU, so while it is useful to discuss the current state of the art in image synthesis, and the techniques they use, it is not feasible to attempt to produce anything of similar size. Therefore it is worth investigating smaller lightweight models and the methods they use.

1.4.1 Crayon (DALL-E Mini)

DALL-E mini, also known as Craiyon, is OpenAI's attempt to address some of the issues that come with large models, it is a VQGAN encoder/decoder model, and although it is significantly smaller than its big brother DALL-E 2, at only 400 million parameters, at this size it still requires one or more high-end GPUs to be used. Even though this model is still too large for the purpose of this project, the fact that its performance comes so close to that of the regular DALL-E 2 while being so much smaller indicates that an implementation of a GAN style model may yield high-quality results for its relative size.



Figure 1.5: Images Generated by Stable Craiyon [24]

1.4.2 TinyGAN

TinyGan is a model created using knowledge distillation from a much larger GAN known as BIGGAN. It was shown by Chang et al. to achieve very similar performance[5], further demonstrating the potential GANs have to perform well at image synthesis tasks even with lower numbers of parameters.

1.4.3 A brief aside on knowledge distillation

Knowledge distillation, first proposed in 2015 by Hinton et al. is the process of reducing the size of the model while maintaining its performance[13]. This is done by taking the large model, known as the teacher model, and removing a given number of parameters, which is often done with the removal of hidden layers, to create a smaller model known as the student model. The goal is then to distil the representations learnt by the teacher model into the student model, allowing it to achieve comparable performance with reduced computational demand.

The main issue regarding knowledge distillation such as in TinyGan is the dependence on a teacher model. In general, teacher models are still large and so still require large amounts of compute to train from scratch. Therefore in the context of this paper, it is not viable, since the aim is primarily to determine whether recent developments in model architectures can be used to train small models from scratch to produce high-fidelity results.



Figure 1.6: BigGan images on top, TinyGan images on bottom [5]

Other similar approaches include 'Network Augemnetation' where a small model is placed inside a larger one, to provide more supervision and show promising results[4]. However, they are again reliant on larger models during training and so are not an option given the aims of the paper.

1.5 Evaluation Strategy

Evaluating the performance of image-generating models is a uniquely difficult challenge when compared to other tasks such as binary classification, due to the subjectivity of the task as well as the lack of ground truth examples for generated images. The difficulty of ground truth comparisons arises due to the inherent diversity of the image data sets the model to learn, as the

output space is very large, containing $n \times n \times 3$ values between 0 – 255 for a colour image of n by n . Furthermore, almost all image generating models use random input to derive the output image, complicating direct comparisons between predictions with ground truth images, as the randomness introduces variability and noise which is impractical to consider during evaluation. In order to address this issue perceptual metrics are often used, such as the Fréchet Inception Distance (FID) and Inception Score (IS). The FID measures the similarity of the distributions between real and generated images based on a feature space learned by a pre-trained model such as InceptionV3 and has been shown to perform well when evaluating diffusion models [26] and GANs [34], a lower FID score is better and models such as DALL-e 2 have a score of 10.83 [3]. The Inception Score (IS) again uses a pre-trained classifier such as InceptionV3 to classify images into categories before calculating the exponential of the entropy of the class probabilities obtained using the classifier.

The subjectivity of the performance of the task is an innate part of most image processing tasks in the field because what is considered an aesthetic or realistic-looking image varies from person to person. Therefore on top of using the FID and IS metrics, I will also create a survey to evaluate the quality of the generated images, as well as the quality and usability of the final application that allows users to interact and use the models.

Chapter 2

Background

2.1 Image processing

In order to understand what approaches are viable and which are not, and why, this section will discuss the fundamental components of image processing neural networks. Subsequently, the section will investigate existing solutions in the field of image processing, to assess what might be applicable to the goal of the paper.

2.2 Convolutions

Image processing with the use of neural networks has changed shape over the years due to the many problems that have arisen in the process. Initially, fully connected dense neural networks were a typical approach when it came to any task involving deep learning, however, this soon led to some issues. In the field of image processing the computational requirements of training dense fully connected NNs are massive, even with the ever-growing power of GPUs, attempting to train large dense networks proves inefficient. This problem is especially difficult in the context of image processing, as the inputs and outputs are very large, for a single 64×64 image, there are $64 \times 64 \times 3 = 12,288$ input values, or, in the case of image generation, 12,288 output values. Consequently, training such networks necessitates managing a vast number of hidden parameters to learn distribution in this image data, which escalates rapidly as additional hidden layers are introduced or the image size is increased.

The solution to this used by virtually all modern image processing architectures, including image generation models, is the convolutional layer. Convolutional kernels operate on the assumption that there is a stronger relationship between neighbouring pixels than far-apart pixels in an image. So rather than having every neuron in an output connected to every neuron in the input, each output neuron is the result of the weighted sum of only the neighbouring input pixels in similar positions. This leads to a greatly reduced number of parameters in a model compared to one comprised completely of fully connected layers, increasing the speed and efficiency with

which it can learn representations of the training data. On top of this convolutional layers are able to take advantage of weight sharing, further reducing their size in parameters. This is carried out by applying a kernel across every region of an image to produce a single output 'image', which can then be repeated using multiple kernels with different learned parameters, to produce an output of stacked 'images'.

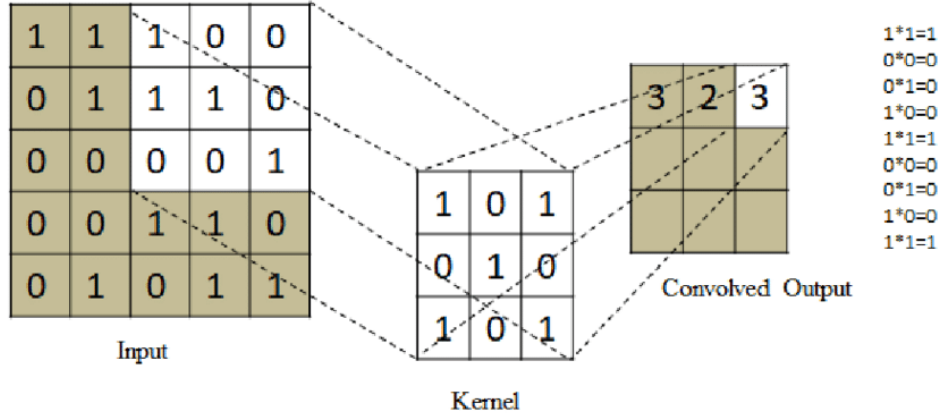


Figure 2.1: Single Convolutional Operation [8]

2.3 Common Image Synthesis Architectures

2.3.1 Flow Models

Normalizing flow models work by transforming a simple base distribution, such as a Gaussian distribution, into a complex target distribution through a series of invertible transformations.

While FLOW models have been shown to perform well in a variety of tasks including image synthesis tasks [23], the transformations between base and target distributions require models with a large number of parameters to perform well. In addition to this, training these models requires computing what is called the Jacobian determinant of each transformation, which can be particularly demanding when dealing with the high dimensional data of images. For this reason, I opted not to move forward with this approach as it is unlikely they would be viable for a model designed to be run on low compute machines. While these have been shown to achieve good results at tasks such as inpainting when used in conjunction with GANs[15] they still are not designed to generate images from scratch which is the largest problem with this approach in the context of this paper.

2.3.2 Variational Auto Encoders (VAEs)

Variational Auto Encoders, or VAEs have been shown to work well at a variety of tasks from increasing image resolution when used in conjunction with GANs[16] to image generation[21].

They are made up of two components, an encoder network and a decoder network. The encoder takes input data and maps it into a latent space representation. Latent space is a lower dimensionality representation of high dimensional data capturing the essential features, which is useful as it reduces the size of hidden layers in models. While the decoder then reconstructs a new representation using this latent space. This is guided by the idea of variational inference, where the encoder learns to approximate the distribution of latent variables given the input data, which is then used by the decoder to learn to generate data samples with similar distributions to the input data.

2.3.3 Diffusion Models

Diffusion models work on the idea of iteratively adding noise to an input image in steps until you have an image where the distribution of pixel intensities approximates a Gaussian distribution [29], then training a model to reverse this process in the same number of steps [25]. The intuition behind this process that makes it such a powerful tool is the iterative and minor changes made at each timestep. The idea is that the model can at the initial timesteps produce the rough ‘outline’ and gradually refine the image at each timestep, being able to add detailed changes in the later timesteps. Because the model only has to make minor changes at each step rather than learning to denoise an entire image in a single operation, it is allowed to focus on learning small-scale transformations that are more effective, which facilitates faster convergence as well as more stable training dynamics, enabling the model to produce high fidelity images.

2.3.4 Generative Adversarial Networks (GANs)

Generative Adversarial Networks or GANs for short are a class of architectures used for image generation in which two models compete against each other to minimize their loss, which in turn increases the other model’s loss. These two models are known as the ‘generator’ and the ‘discriminator’. The purpose of the generator is to produce outputs similar enough to the training data to make the discriminator perceive them as part of the training set rather than generated. The goal of the discriminator is to correctly classify inputs into two classes, real (part of the training data) and fake (created by the generator).

2.3.5 Why GANs and Diffusion

While Variational Auto Encoders and Flow models are state of the art approaches that have been shown to perform well at image synthesis tasks, I have opted to focus on the use of GANs and diffusion models to generate images for a number of reasons.

Firstly GANs and diffusion models have been shown to be able to produce much higher fidelity samples [22], compared with VAE models relative to their sizes. As the final outcome

of the project is to create something that can output images on low power devices, keeping the size of the model small is essential. Furthermore, VAEs struggle with inference speed and generally require longer training times relative to their size [1]. In addition, they often struggle to generate sharp and high-quality images. This is due to the inherent trade-off between reconstruction accuracy and latent space smoothness.

The choice to exclude flow models for this paper is primarily down to the computational cost of training and using them. While they are capable of producing high-fidelity results, the size of the model means that to most people they are not accessible to use or train on a local machine.

On top of this, although flow models and VAEs are often used for image synthesis, and are technically capable of doing so, they are not generally well suited to generating images from noise or other random input. Rather they perform best at image augmentation, such as inpainting or editing. On the other hand, GANs and Diffusion models were first proposed as techniques for image generation, and have been demonstrated innumerable times to be simple and effective at this task.

Implementing both a GAN model and a diffusion model will be useful as it will allow for comparison between the techniques used in the latest models such as DALL-E 2 and older ones. Doing so will show whether there actually is a marked increase in performance using these techniques in small models.

2.4 GANs in Depth

The type of GAN to be used in this project is the Deep Convolutional GAN (DCGAN), which refers to the architecture of the model, being comprised of many convolutional layers between the input and output layers. As mentioned previously the model that is trained is made up of a discriminator and generator, which can be viewed as two separate models in their own right.

2.4.1 The Generator

The convolutions used in the generator are slightly different to the standard ones described earlier in the paper, as rather than reduce the dimensions of the output they increase it, this is called deconvolution. As seen in the illustration, they work by taking in a single value and multiplying that by a weighted kernel, and then appending it to the output, resulting in a larger output dimension than the input dimension.

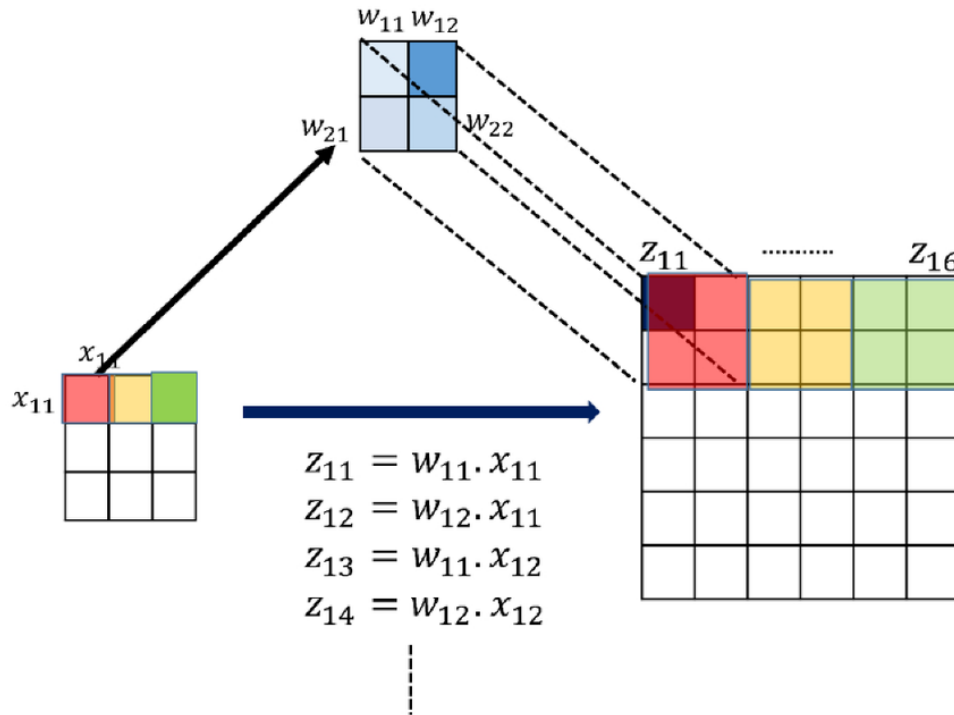


Figure 2.2: Deconv Operation [36]

In order to add randomness, so as to prevent the generator from outputting the same value for each prediction, its input is a vector of random values sampled from a Gaussian distribution. This is then what is used as input for the deconvolutional layers. The deconvolutions are applied sequentially starting from a random input, generally of size 100, until you have the desired output dimensionality.

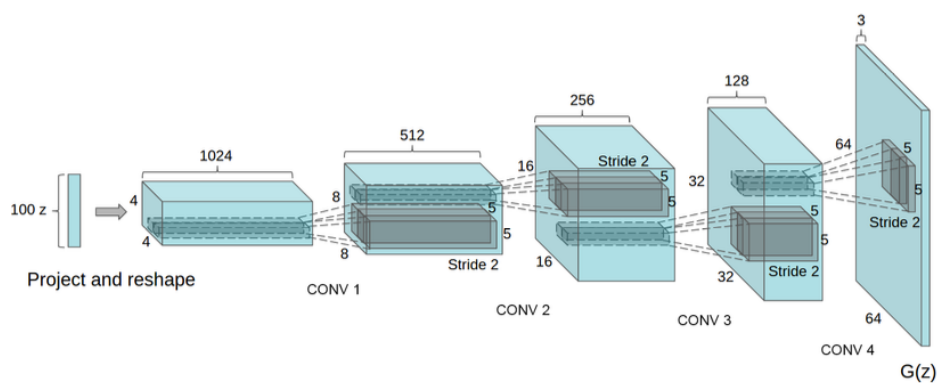


Figure 2.3: Generator Overview [19]

2.4.2 The Discriminator

The task of the discriminator is to take an input image of $n \times n \times 3$ and output a single value between one and zero, representing a prediction of real or fake respectively. To do this DCGANs are comprised of several convolutional layers. The key insight behind the use of the discriminator being a core part of the GAN is its capability to capture the underlying distribution of the training data. This means that rather than using actual images as feedback to train the generator network, which is flooded in a number of ways, instead, it is the discriminator's feedback that guides the generator's learning process.

2.4.3 Training Process

Training process

Generally, the weights of the discriminator are initialized with random weights before the training process, which can be broken down into several steps:

Generator step

The generator takes a batch of random noise samples and generates a fake sample for each random noisy input

Discriminator step

The discriminator is given as input this batch of fake samples, and an equal number of real samples from the training set, in order to classify each sample as either real or fake.

Discriminator Loss

Because the discriminator is performing a binary classification task, binary cross entropy (BCE) is the most appropriate loss function.

$$BCE = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)$$

where:

y_i is the label

$\hat{y}_i$ is the prediction

The intuition behind this function is to produce a larger loss value when the prediction and label are very different, and a small one when they are similar. The weights of the discriminator are updated separately to the weights of the generator.

Generator Loss

This is a more complicated process as it is dependent on the backpropagation process of the discriminator. Because the generator and discriminator are adversarial, the generator has the goal of maximising the discriminator's loss, which uses binary cross entropy.

$$BCE = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)$$

where:

y_i is the label

$\hat{y}_i$ is the prediction

Since the label, y_i , will always be 0, as the generator is being trained and all training examples are fake, the left-hand term can be removed, giving the equation:

$$Loss = \frac{1}{N} \sum_{i=1}^N \log(1 - \hat{y}_i)$$

During training, this loss is calculated using the discriminator output, and the backpropagation is performed through the discrimination and generator, as if they are stacked to create one large model. The discriminator's weights are frozen so that only the generator model is updated.

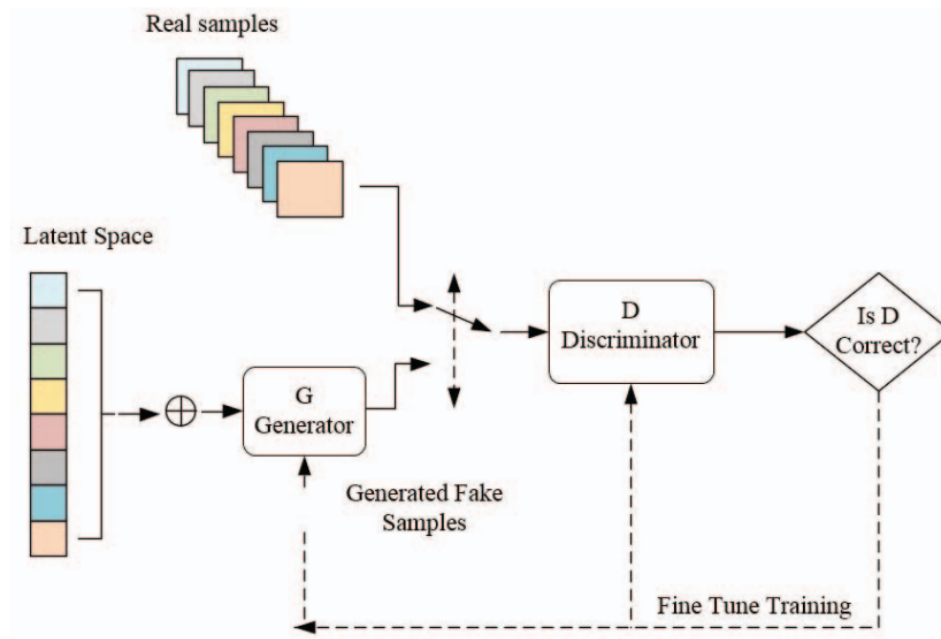


Figure 2.4: GAN Training Overview [11]

2.4.4 Diffusion Models in Depth

U-Net Architecture

The typical architecture used in diffusion models is the U-net, named after the shape it resembles. The U-Net follows an encoder-decoder structure, whereby the size of the hidden layers is reduced, reaching its lowest dimensionality in the middle layer of the U-net, before being upscaled to the output dimension. This is used in conjunction with skip connections, that connect corresponding encoding and decoding layers. The main benefit of the U-net architecture is the reduction in the number of parameters compared with an equivalent standard convolutional net, reducing the number of weights that need to be updated each training step, thus making it promising when considering the want of keeping the number of model parameters low.

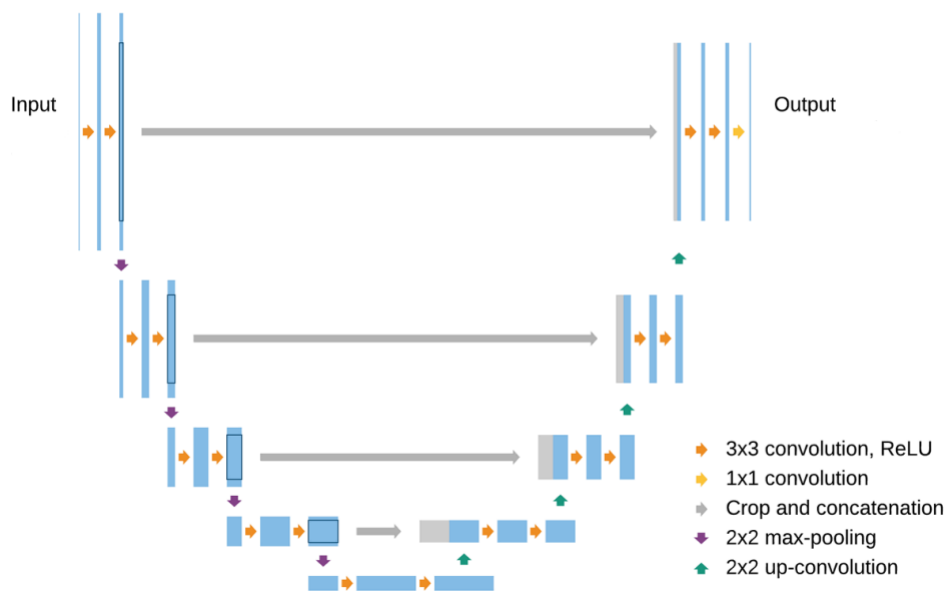


Figure 2.5: U-Net Overview [28]

Training Process

The aim of a diffusion model is to take a sample image made up of Gaussian noise and iteratively remove the noise in n time steps until you have the final image. This means there are two processes involved in training a diffusion model, one for applying noise to images on which the model can learn, and the standard gradient descent used to update the model weights.

Forward Diffusion Process

Assuming you have t denoising timesteps, one step of the forward diffusion process is given as

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t, \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

where:

$\mathcal{N}$ is a normal distribution

$\sqrt{1 - \beta_t}x_{t-1}$ is the mean

β_t defines the variance

Beta is derived from what is called the schedule, in other words, the number of diffusion time steps. A higher (later) value of t means more noise and less of the image and vice versa.

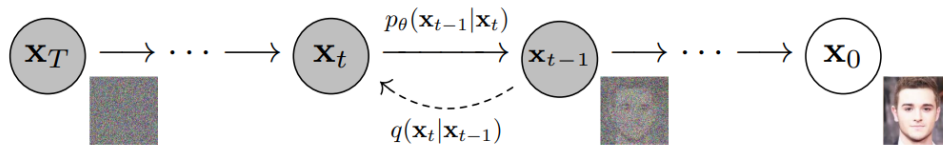


Figure 2.6: Noise Operation Graph [14]

Backward Diffusion Process

This is the part in which the model is given an image at timestep t with an amount of noise dependent on the given time step t , and is asked to predict the time step $t + 1$, after which the standard gradient descent algorithms are used to update the model's weights.

Chapter 3

Methodology

In this section, the specific techniques and model implementations are discussed. In addition, the requirements are assessed, including data selection and training hardware. Hyperparameter testing and selection to maximise model performances are also described.

3.1 Hardware

Although the aim of the project is to assess the potential to train models on home devices, in order to speed up the development of the project, a GTX 1060 3GB was used to train these models. This allowed for faster prototyping of implementation with varying hyperparameters. Although a GPU was used, it did not dramatically improve training time. When training on a CPU (I7 6700k) one epoch took 11 hours, whereas when using the GPU, it took 4 hours. This means that although the training process would take longer than what was used in this paper, it is not a dramatic change. For comparison, GPT3, of similar size to DALL-e was trained for 34 days on 1024 GPUs. Relative to that, the training process of the models being implemented will be much faster and more accessible.

3.2 Latent Diffusion U-Net Implementation

As mentioned previously one of the architectures most commonly used in conjunction with image synthesis tasks with diffusion are U-nets. While the overarching structure follows the same idea, there are still many variations in design to consider.

3.2.1 Residual Blocks

Residual blocks refer to the encoding and decoding layers, the set of operations applied to input to process and scale up or down. These can include what are commonly known as 'down blocks' and 'up blocks', which use what are known as residual connections.

3.2.2 Down Blocks

The role of these blocks is to reduce the dimensionality of their inputs, while preserving high-level semantic information, by producing what is known as a latent representation. The reduction in output space compared with input space reduces the number of model parameters compared with a standard deep neural network.

3.2.3 Up Blocks

Once the downsampling blocks have produced a latent representation, the purpose of the up blocks is to use the information stored in the latent representation and create an image. This means gradually applying deconvolution layers to the latent representation in a similar way to the method used by a generator in a GAN.

3.2.4 Residual Connections

Residual connections are a critical part of the U-Net architecture that drives its performance. The residual connection links a down block to its corresponding up block (usually there are equal numbers of down and up block that use the same scaling factor). This leads to several advantages;

Vanishing Gradients

While deep neural networks can learn complex representations of data distributions, adding depth leads to the vanishing gradient problem. This is where the gradients of the loss function approach zero when moving through the backpropagation process, due to weights commonly being close to zero. This means that the model, specifically in the early layers, is not able to learn and update efficiently. This can cause a model to underfit or converge slowly. Residual connections offer a solution to this by creating a pathway between distant layers through which gradients can 'pass'. In essence, it allows information about a model's performance and the way its weights should be updated, to reach early layers more effectively.

Spatial Representation

As a network downsamples the input image, spatial information is gradually lost to pooling operations. Residual connections allow for the preservation of this information by passing it directly to the upscaling blocks, alongside the semantic representation.

To sum up residual connections are a key component of the U-Net to facilitate efficient training and to allow the model to learn a good representation of the data.

Residual Block Implementation

The attention mechanism first proposed in [30], has been shown to be a powerful technique not just in text-based tasks but in many fields and they have also been shown to work very well with U-Net architectures [31]. In the context of image synthesis models, they are able to capture spatial dependencies between different sections of the image. This is an advantage over standard convolutional kernels which usually have small receptive fields, since attention matrices can represent long-range spatial dependencies across the entire image.

Used in conjunction with the architecture of the U-Net, self-attention could be leveraged to utilise the semantic representation and spatial representation as query and key values respectively [7]. This likely would come with many benefits such as training stability, and improved performance. The major drawback to this however is the computational cost they come with, especially when considering the size of the images. The computation of self-attention involves pairwise comparisons between every position of the input image, specifically between each position and every other position. This results in a complexity of $O(n^2)$. This means that the resources required to train and use models with this mechanism very quickly become prohibitively large. Although using the attention mechanism would likely lead to better performance, it would also mean a much bigger model, because the project goal is to produce lightweight models this is unlikely to meet the goal of the project.

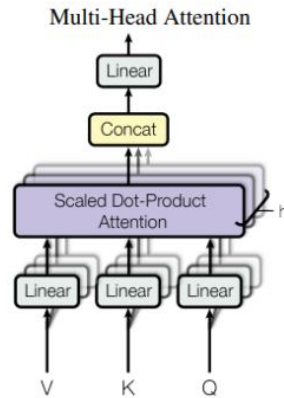


Figure 3.1: The Attention Mechanism [30]

The incorporation of residual connections in a U-Net can be done in several ways, however, to reduce the computational cost, the approach I will use is a simple concatenation of the residual and latent representations before being passed to an up block rather than calculating attention weights. These have been demonstrated to achieve high accuracy at tasks such as image segmentation [28] but can also be used for image generation.

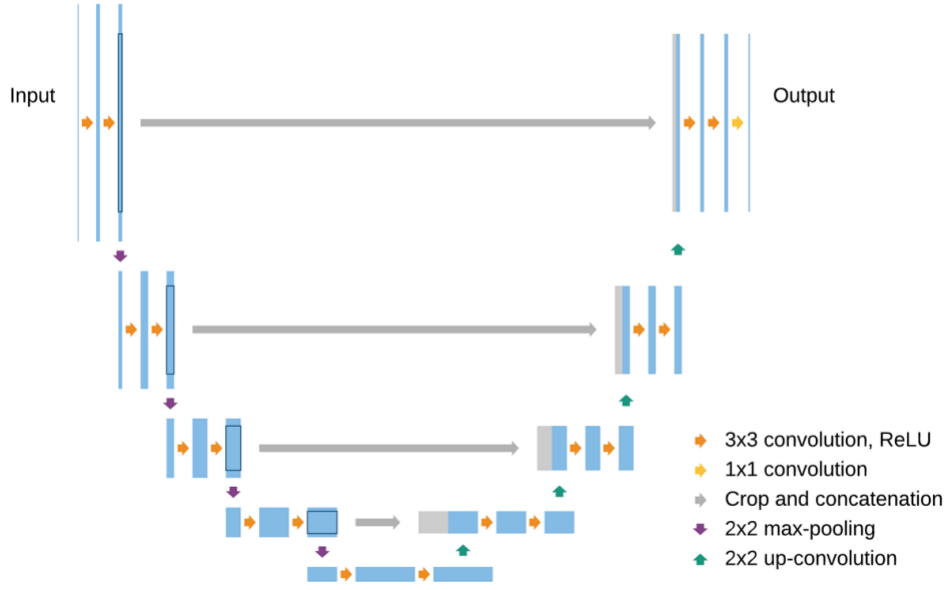


Figure 3.2: U-Net Residual Connections within a U-Net [28]”

3.2.5 The Final Residual Block Design

The authors of the paper ‘Semantic Image Synthesis via Diffusion Models’ [32] demonstrated a model using relatively simple residual blocks that can achieve high-fidelity images, with the resulting model achieving an FID score of 27.5. The main drawback to this design is the use of attention blocks throughout, due to the aforementioned high memory demand making them unfeasible on a machine without a large amount of RAM. Therefore a similar model may be a reasonable approach whereby the attention blocks are removed. One issue with the removal

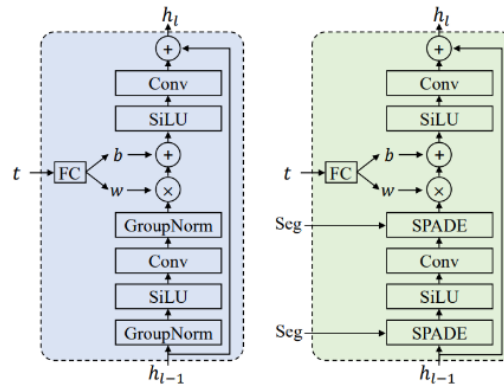


Figure 3.3: Encoder and Decoder Blocks described in Semantic Image Synthesis via Diffusion Models [32]

of the attention blocks is the size reduction, and therefore complexity of the model. Although the aim is to allow the final model to be run on any device, too small of a model will lead to under-fitting and poor performance. To prevent this it is important there is a sufficient number of parameters in the layers of each block.

Based on the above proposed block, which takes a time parameter of the current time in the diffusion process, this is the simplified block created for this project:

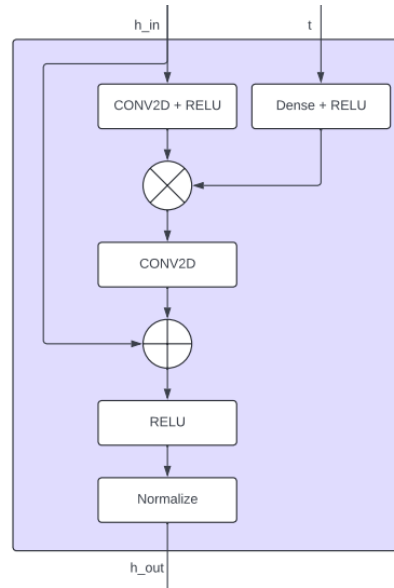


Figure 3.4: Implementation of residual block used for this project

These layers will then be stacked with max-pooling layers to give the resulting U-net:

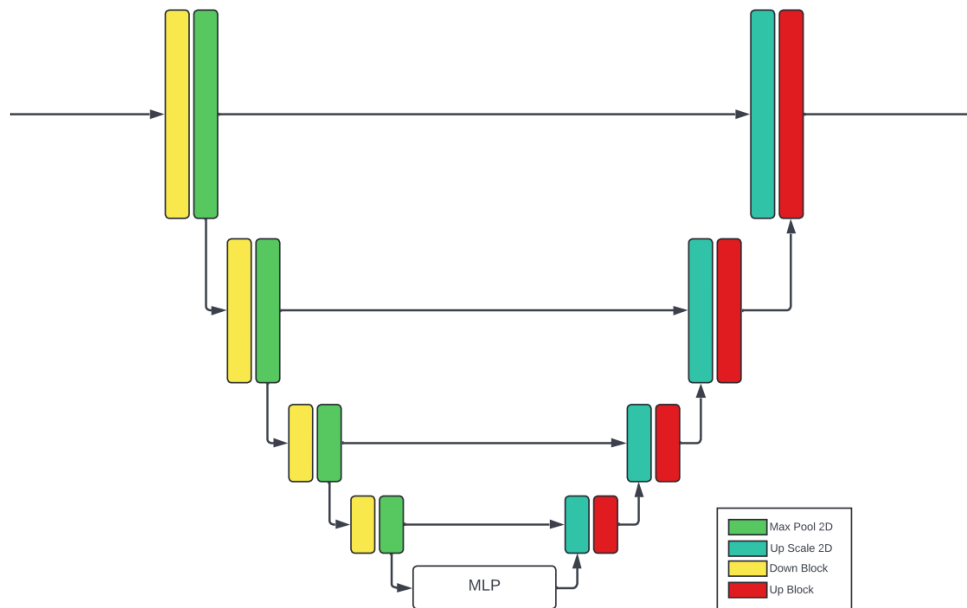


Figure 3.5: Implementation of residual block used for this project

3.2.6 Time Schedule

While a larger number of timesteps can help to increase training stability and overall performance of the model it comes with drawbacks:

More Training Examples

While often this is a benefit, it also means per epoch, you must show the model more examples, in this case, the number of images $\times$ number of time steps. This will increase the training time of the final model and lead to increased computational resources being required.

Longer Generation Time

The second key drawback to using large numbers of denoising steps is the time to produce a final image, compared with a standard approach of one-shot image synthesis. Especially when very large numbers of timesteps are used, on the order of 1000, as demonstrated by Franzese et al[10], may mean that on a low compute machine the generation process is inhibitive slow.

While reducing the number of steps then is important within reason, at the limit (say, one step), you lose all benefits of the diffusion process. Therefore it is important to find a balance between quantity and quality. The number of steps I selected was 25, as this yielded good results while keeping training and generation time low.

Time Shedule Shape

The shape of the time schedule describes the rate at which an image is transformed into Gaussian noise with respect to t . There are three main approaches to this, the step function, the exponential noising, and linear noising.

Exponential Noise Schedule

This scenario describes an exponential growth towards a Gaussian noise from an image.

Step Function Noise Schedule

In this case, noise is introduced at a constant rate, until a given time step, where the beta coefficient may abruptly change, altering the rate at which noise is added.

Linear Noise Schedule

a linear noise schedule the noise is increased linearly with respect to t

Diffusion models are still in their infancy and certain intricacies in aspects such as the time schedule are not fully understood. There is no clear evidence that one is better than another suggesting that the shape of the noise schedule was not a significant factor in a model's performance, provided enough timesteps were used to generate images. As the linear schedule

is the simplest, while achieving equal performance, this is the schedule I will be implementing to train the model.

3.2.7 Diffusion Loss Function

The loss function I will use to train the diffusion model is the mean squared error loss (MSE). This is a simple and well-understood loss function in the field of deep learning and is applicable to image processing tasks, because it can be calculated on a pixel-by-pixel basis.

$$MSE = \frac{1}{N} \sum (y_i - \hat{y}_i)^2$$

3.3 GAN Implementation

The deep convolutional GAN proposed by Radford et al [19] is a simple and relatively small model in terms of the number of parameters it has, so I will be implementing a model design inspired by this, in order to use it for comparison with the latent diffusion model which uses state of the art techniques.

Why Just Colour Images?

Due to the complexity and size of even relatively small models by today's standards, it is very difficult to understand what is happening within a neural network, including any performance issues. In this case, the models are being built to generate colour images, however, this may lead to issues associated with the complexity of three colour channels compared with the single channel that black and white images have. Because the models are being designed to be small so as to be able to be used on lower power machines, they may not be able to generalise well to the complexity of colour images. So for comparison, I will create a separate model of equal size that will be trained to generate black and white images. This will be a useful tool to help understand what may be attributed to the performance of the colour models, be it good or bad. For example, a much higher performance of the black and white model may suggest that the colour models are underfitting due to a lack of parameters. The goal of this model is to elucidate the features that make work and those that don't, and should give some direction to continue in should the research be continued beyond the end of the paper.

3.3.1 Generator Implementation

As shown by Manisha et al [18], to allow a model to generate a diverse range of images, it is crucial to have a high enough input dimensionality. Based on what was shown roughly 100 input values using a Gaussian distribution is sufficient for a dataset containing 200k images.

Therefore as input, the generator for this project will take 100 random values samples from the Gaussian distribution.

This will be followed by a series of deconvolutional layers to upscale the output to a dimension of $64 \times 63 \times 3$ for the colour model, and $64 \times 64 \times 1$ for the black and white model, the dimensions of the images being generated.

3.3.2 Discriminator Implementation

The discriminator model will also be a deep convolutional neural network. It will use a series of convolutional layers to gradually reduce the dimension of the original input to an output of 1 value, between 0 and 1.

In conjunction with the convolutional layers, drop-out layers will be used in between each conv layer. Dropout is an essential part of the training process of GANs. Applying dropout to a model while training means that when making a prediction, the model has a certain percentage of its outputs (neurons) set to zero in each layer. The reason that it is so critical while training a GAN is due to a discriminator being able to overfit to a training set - meaning it has learnt exactly what images are in that set. This then generally results in a generator network also overfitting by then learning to only generate a subset of the images in the training set to a very high accuracy. This is called mode collapse and is one of the most difficult problems to avoid when training a GAN. The standard dropout rate used when training GAN is 0.1 to 0.5, initially, I will use a dropout of 0.2, and will alter this if evidence of mode collapse becomes apparent.

3.4 Requirements

3.4.1 Dataset

Although it is not reasonable to expect to be able to train a model that is much smaller on such a complex dataset as the ones used to train DALL-E, Midjourney and the like, it is still important to select one with enough complexity to sufficiently demonstrate whether locally trained models can learn complex representations in order to generate a diverse range of images. In the paper by Feng et al [9] it was demonstrated that training GANs on a dataset above a size of 8000 samples does tend to suffer from what is known as GAN replication, caused by various factors such as mode collapse previously discussed in this section. Where the model learns to predict nearly exact replicas of images in the training set. Therefore it is vital to use a data set with at least 8,000 samples

With all these factors in mind, the dataset I will use is the Animals10N Dataset: containing 10 classes of animals, with 50,000 images in total, of 5,000 per class.

3.4.2 Data Preprocessing

Because the training data is images, there generally is not too much that can be done to remove such things as outliers without manually reviewing the data set by hand, which is not feasible. The majority of the processing is done to enhance model convergence and ensure the images are compatible with the model training process.

The main two operations were resizing and normalization. The resizing ensured the training images were the same dimension as the model's output, so they could be used as training examples.

Normalization is the process of scaling the intensity of every pixel to between zero and one. This is done such that the scale factor sets the brightest pixel in the original image to one, and the dimmest pixel to 0, all other pixels are also scaled by this factor. Normalisation can lead to faster convergence because it means the scale of input data is homogenous, rather than making the network learn from a set of images with vastly different max and minimum values. This allows the model to focus on the underlying distribution of data rather than the above-mentioned variations that may have otherwise been present.

3.5 Training

Due to the inherent complexity of modelling a distribution of image samples, the training process of image generation models often requires more epochs than other tasks in the field such as classification tasks. The time it takes for a model to converge to the best solution is also dependent on many factors including its size and architecture, therefore there is no clear way to determine the training time required for the proposed models. To address this, every fifth epoch the model will be evaluated using an evaluation set, until marked improvement stops, at which point the training process will end, and the best-performing set of weights will be selected for the model.

Although the Inception Score and Fréchet Inception Distance are much more representative metrics of a model's performance than simple loss or accuracy, I will be using loss at the end of each epoch for evaluation. This is because the IS and FID require large numbers of test and prediction samples, usually around the order of 1,000 or more. Performing either of these evaluations will significantly reduce training speeds and may reduce the total number of epochs I am able to run, which may in turn prevent a model from finding its optimum set of parameters.

3.6 Application Design

An effective application design is important because it directly influences user satisfaction and experience. This can be split into two parts; what can it do, and how easy is it to use?

3.6.1 Functionality

For the application to be functionally useful it must be able to perform the following tasks of:

- image generation
- image saving
- ability to load previously generated images

3.6.2 User Interface

One of the main aims of this project is to make more accessible the ability to use high-performance models on any device, thus providing an application with a user-friendly interface that can be easily downloaded along with the models is critical. This eliminates the need for every user to use Python scripts to call an API for example. Furthermore, as shown by Wijaya et al, the user interface has a large impact on how the user views images[33], and while they demonstrated this for art viewing applications, there is a large overlap with the aims of this application as they both represent images. To achieve this, the UI will be clear and easy to use so that it encourages use from all demographics, regardless of age or experience with computers. Furthermore the UI will be kept minimal and simple so as not to overwhelm end users. To keep it simple the User Interface will contain a single homepage acting as a central hub. The homepage will be comprised of two main sections, with a section displaying previously generated images, and a section containing 3 tabs, each containing an option to generate images using one of the 3 respective models. This will allow users to easily switch between each of the image-generating tools, as well as look back over what they have already created.

With the resulting user interface looking as follows:

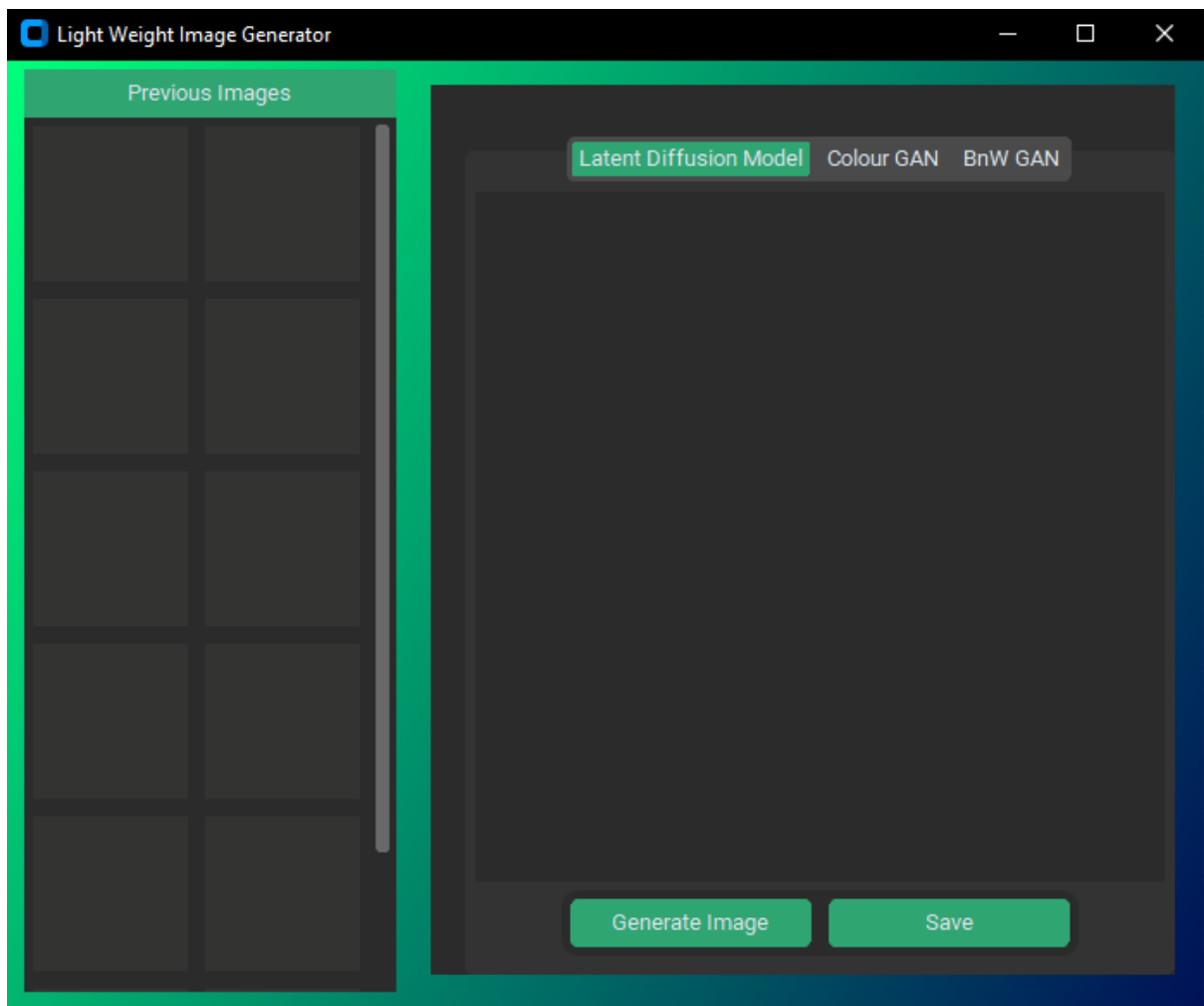


Figure 3.6: Final User Interface Design

Chapter 4

Evaluation

The assessment of the final artefact produced by the project can be split into two sections, the performance of the application, and the quality of the image generation. By thoroughly evaluating both of these components, the success of the project can be better understood, and a clearer direction to move in is available if the project were to continue.

4.1 Model Evaluations

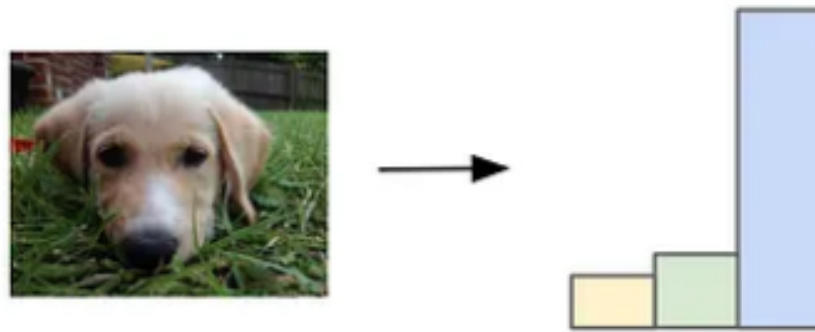
Comparing image generation models can be challenging, since there are many factors that can affect a given measure, such as resolution and test set quality. However, two of the most commonly used measures are the Inception Score and the Fréchet Inception Distance. These metrics are quantitative measures and have been used to evaluate the majority of existing state of the art image generating tools, including diffusion models. This will allow for a direct comparison between the models created for this paper and the latest publicly available ones. Being able to directly make this comparison will allow for a more complete and comprehensive evaluation of the models I create, and the possibility of creating lightweight high-fidelity image generation models from scratch.

4.1.1 Inception Score (IS)

The inception score, first proposed in the paper 'Improved Techniques for Training GANs' [27] has been shown to be a powerful tool for training and evaluation of image generation tasks. This metric assesses both image quality and diversity, making it a vital tool in evaluating the performance of a model. The key part is the diversity, as this metric prevents a model suffering from mode collapse from receiving a high score. A pre-trained classification network is used to classify each image it is given, producing a probability for each class where the sum of probabilities adds to 1.

Image quality

The intuition behind this part of the equation is that if the image is of high quality the classifier net being used should confidently be able to classify the image. Resulting in a predicted probability distribution that looks like the graph below.



Ideal label distribution

Figure 4.1: Label Distribution [17]

Which is represented by the following formula:

$$\text{Class Probability} = P(y|x)$$

where:

y is the class probability predictions

x is the image input

Image Diversity

The diversity of the model's predictions is represented by what is known as the marginal distribution. This is done by taking the sum of the predicted probability distributions over many predictions. The ideal marginal distribution should look like an even distribution, meaning that the model can produce images in a variety of classes, rather than being biased to just a few.

$$\text{Marginal Distribution} = P(y)$$

where:

y is represents a given class

Different labels sum to give uniform distribution

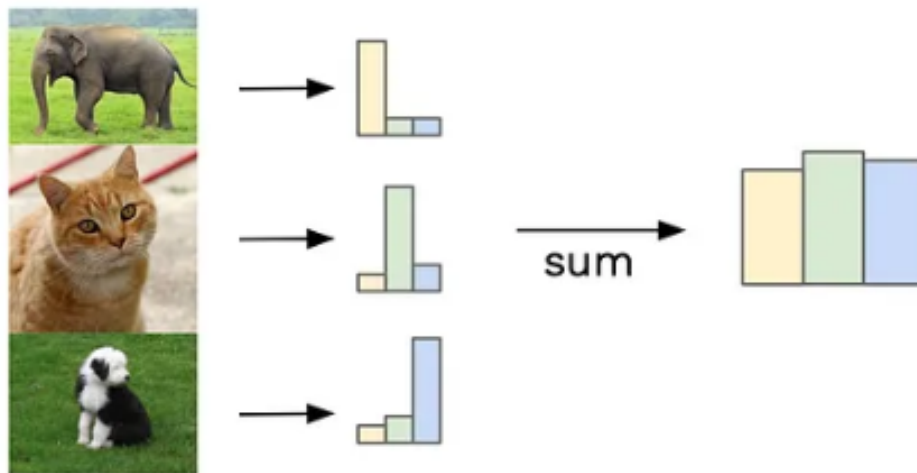


Figure 4.2: Marginal Distribution [17]

To calculate the Inception Score we want to compare the marginal distribution to each label distribution using what is known as KL divergence, and take the exponent of the average. Because the ideal shape of the marginal distribution and the label predictions are different, a larger KL divergence indicates better performance, as shown in the graph below.

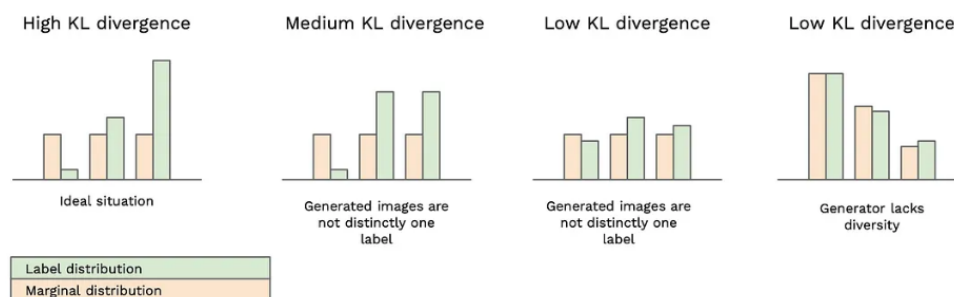


Figure 4.3: KL-Divergence [17]

Fréchet Inception Distance (FID)

This metric focuses more on the similarity of distributions of the training set and generated images. While the IS can give measures of diversity and quality, it cannot represent how similar the generated images are to the real samples, this is what the FID remedies. The mean and the covariance of the feature representations of both the generated and real image sets are calculated. The Fréchet distance between the two sets is then calculated using their respective means and covariances.

Initially, much like the IS, a pre-trained network is used, but in this case to extract feature representations of the real and generated images.

The following formula is used to calculate the FID:

$$FID(X, Y) = ||\mu_X - \mu_Y||^2 + Tr(\sum_X + \sum_Y - 2\sqrt{\sum_X \sum_Y})$$

where:

μ_X is the mean of set X

$\sum_X$ is the covariance of set Y

Tr represents the trace of a given matrix (the diagonal running from top right to bottom left)

This results in a metric that quantifies the dissimilarity between two distributions. The lower the score the more similar the distributions are, and in this case, the higher quality the images.

Qualitative Analysis

While the IS and FID have proven to be very representative metrics of performance for image synthesis models, they are not perfect. Both metrics are dependent on the quality of the training set, as well as the number of test examples available, which can influence the measure of diversity they give. In other words, the scores are heavily reliant on the test set they use and cannot be viewed as concrete evidence for any conclusion. Because the datasets used for my models and the large models being used for comparison, it cannot be a perfect comparison in performance. Therefore I will be using a survey to evaluate the performance of each model as well.

This will be used to assess:

- Realism of the images
- Diversity of the images
- How fast was the model at generating images
- How consistent was the performance of the model

Participants of the survey will be asked to rate each of these between 1 and 5.

4.2 Model Performance Review

Although understanding the inner workings of large neural networks is not feasible given the size of even relatively small models, it is still important to have a holistic overview, of what may have caused issues, providing future paths for investigation that may yield better performance. This can be achieved with the aid of empirical data and a good comprehension of the

underlying mechanism on which the networks are based. This section discusses the results of the performance of the model and user interface. The goal is to evaluate and understand the reasons for its performance, measured against the predefined criteria. Doing this will provide direction for development if the project were to continue and in addition, inform the conclusion as to whether low-power machines really can be used to pre-train AI image generation tools from scratch.

4.2.1 Quantitative Performance Review

Table 4.1: Model Performance (In bold are the models created for this paper)

Model	Fr�chet Inception Distance	Inception Score
Diffusion Model	524.7	1.42
Colour GAN	563.4	1.21
BnW GAN	632.9	0.92
LDM-4	10.56	103.49±1.24

4.2.2 Model Survey Results Analysis

This section discusses the scores achieved for each question of the survey and the potential reasons for the performance on each question. Each question was asked for each model.

Image Realism

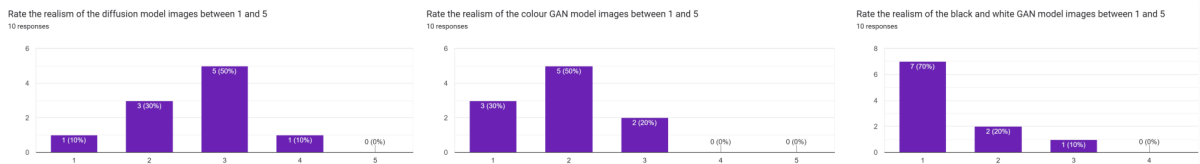


Figure 4.4: Feedback on Realism of Images

This is the most important criterion in this paper, if this is not achieved it shows that techniques used today are still dependent on large amounts of computation that many do not have access to. All three models performed relatively poorly in this section, with the coloured GAN and diffusion approaches doing better than the black and white GAN. This performance is reflected in the FID and IS scores as well.

One of the main contributing factors may be the nature of the training sets and the difficulty of the task of image generation. One might think that because the complexity involved in learning the distribution of the coloured images is greater than that of the black and white images, the BnW model would perform better. However, as demonstrated, this was not the case, it is possible that although the coloured images are more complicated to predict, the extra colour

channels provide enough additional information to allow users (people that were surveyed) to perceive objects even when the fidelity of the image was poor. For example, being able to distinguish between foreground and background based on colour even if both have equal brightnesses. This is not the case in black and white images, this suggests that for these models to be able to generate interpretable images, they have to learn a more accurate representation of the underlying data set.

While the performances are similar for all three models the diffusion model does outperform the other two models in both the survey and quantitative evaluation. This suggests then that the technique of gradual denoising of an image in many steps, as opposed to in one shot as the GANs do is superior in creating high-fidelity images. This is further supported by the fact that all the models are roughly the same size in terms of parameters (as all are within the same order of magnitude).

The lack of observed performance of the models is indicative of underfitting, where the model has failed to learn the underlying distribution of the data properly. Usually, this occurs because of either a lack of model complexity, lack of training examples, or over-regularization.

In the case of the GAN:

It is unlikely due to over-regularization, as the only kind used was dropout in the discriminator, and there was no evidence of mode collapse during the training process meaning the training process was stable enough for a model to learn sufficiently. This problem does not apply to the diffusion model as they do not suffer from mode collapse, since they are not adversarial and instead use supervised learning.

Thus it is more likely a combination of insufficient training data and lack of model complexity. To address these issues when training a model, would necessitate creating a larger model, as well as more training examples. Both of these solutions lead to the use of more compute, in the form of RAM to store the model during training and generation, as well as longer training times to process the larger number of samples.

In summary, it appears that while it is possible to train models that can produce high-quality images, doing this using low numbers of parameters is not possible with existing techniques. Requiring huge amounts of training data, and very complex models.

Image Diversity

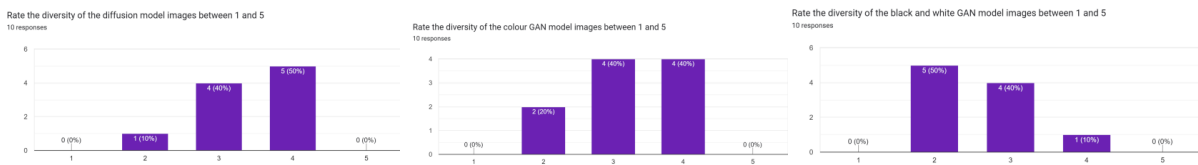


Figure 4.5: Feedback on Image Diversity

This section performed well in the survey, and based on visual inspection, all the models

generate diverse ranges of images.

In part, this suggests that the dataset used to train the models contained enough variety in its images to allow each model to generalise to a large distribution of data. This is also evidence that the training process of the GAN did not succumb to the problem of mode collapse, in which it learns to imitate just a few images.

In comparison with the current best solutions, however, the diversity is much worse. This in part is down to the sizes of the datasets they were trained on. DALL-e 2 was trained on the dataset 'WebImageText', containing over 400 million images, and Stable diffusion was trained on the set 'LAION5B', containing 5 billion images. This additional size allows the data set to represent much more complex and varied data distributions, which, in turn, allows to model to learn to generate a much wider range of images. Put simply, there was more content for these models to learn from, than the ones created for this paper. This represents a real problem when training image synthesis models, as for a given class, or type of image, generally you need hundreds or more training examples [35]. To train a model that has a diversity on the level as the models discussed above necessitates extremely large numbers of training examples. In the current field of Image Processing and in the wider field of artificial intelligence there is still little in the way of a workaround for this issue.

Generation Time

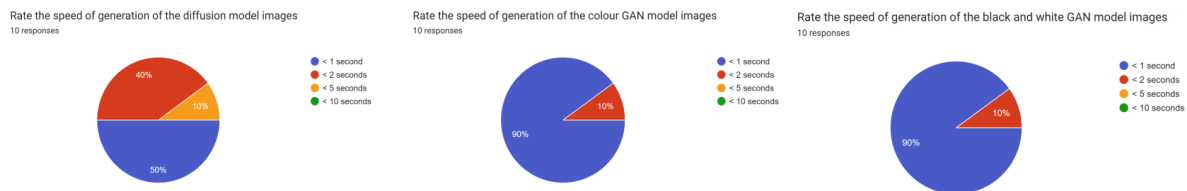


Figure 4.6: Feedback on Generation Speed

This criteria was met well, scoring high in the survey.

All the participants rated both the GAN models at under two seconds. The Diffusion model had a generation time rated at under five seconds by all participants, and under two by most which is also reasonable when compared with the times large diffusion models require.

Why is the diffusion model slower?

The diffusion model is nearly twice the size of the other two models, however, this is not the reason for such a difference in generation time between the models, as the generation time is over twice as slow. It is due to the process the diffusion technique uses, refining an image over multiple timesteps. This requires the model to be called t times, where t is the number of time steps. In this case that is twenty-five calls of the model compared with the single call needed for the GANs to generate images.

While the generation time of these models is faster than the recently released models such as DALL-e and Midjourney, they are also much smaller. On top of this, the output resolution is significantly lower, where DALL-e 2 can produce images between 256×256 and 1024×1024 the ones created for this paper have output dimensions of 64×64 . This means they sacrifice both resolution as well as fidelity as discussed in the section 'image diversity', so although it is clear that GANs outperform diffusion models in terms of generation time, there is no actual performance difference relative to the number of parameters.

In addition, it is worth considering that while the GANs generate images faster than diffusion of the same size, the quality of images is lower. To achieve comparable performance, a GAN will need more training data and more parameters, negating the advantage of faster generation.

Consistency of Quality of the Images

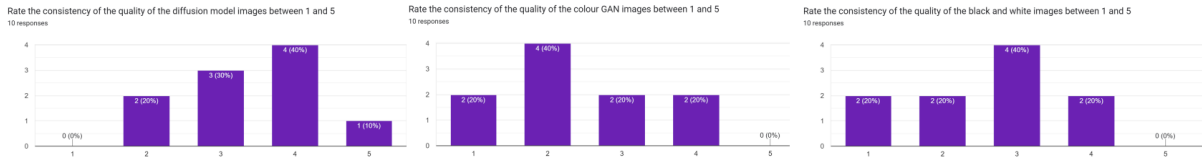


Figure 4.7: Feedback on Consistency of Image Quality

This metric was met with mixed feedback for all versions, each achieving a diverse range of scores. Looking at the results it is clear that while they can all create ok-looking images, they also fail in some cases to create images with any identifiable features.

The GAN models were rated much more inconsistent than the diffusion model. Their inconsistency is indicative of two problems, an unbalanced dataset and model underfitting. Regarding the imbalance of the dataset, I refer to the quality of the images. Suppose certain categories of images are of lower fidelity than others. In that case, it will lead to the models being unable to learn accurate representations of the data in those classes compared with classes with high-quality samples. In conjunction with this, the discriminator may not have been complex enough to learn a general representation of the entire dataset. As there are ten classes of animals making up the samples, the model is required to learn a representation of each class. The inconsistent image quality suggests that it has learned to represent some classes to a high accuracy while not being able to fit to the entire set. This combination may have led to a situation in which a model lacking the complexity to fit properly to the dataset learned to prioritize the classes of higher-quality images in order to minimize its loss.

On top of this, while the classes were balanced in terms of the number of images, it may be that the nature of some image classes is more complex than others. The dataset was large, containing $\sim 50,000$ examples, but this still only leaves about 5000 images per class. For some classes this may not have been sufficient, further incentivising the model to disregard learning

to represent simpler classes, or leaving the model unable to learn the class' data distribution accurately.

In contrast, the diffusion model did not suffer anywhere near as much from these issues. This is because while the GAN generator uses unsupervised training, diffusion models directly model the likelihood of the data. As mentioned above, this can result in a GAN model only producing certain classes, since it only has to fool the discriminator, whereas in diffusion models, training is supervised with the use of image samples during backpropagation. Because of this, the diffusion model is forced to learn a representation of the entire dataset to minimize its loss.

4.3 Image Examples

below is a set of nine images generated by each of the three models.

4.3.1 Black and White General Adversarial Model Images



Figure 4.8: Set of BnW Images

4.3.2 Coloured General Adversarial Images



Figure 4.9: Set of Coloured GAN Images

4.3.3 Diffusion Model Images



Figure 4.10: Set of Diffusion Model Images

4.4 Evaluation of the Application

Since it is not possible to perform a quantitative analysis of an application's user interface, I will create surveys to assess the final UI design.

This will be used to evaluate the performance in the following categories:

- Learnability - How easy the User Interface was to understand beginning from a place of zero knowledge
- Efficiency - How well the User feels they can navigate the application without unnecessary interactions

- Aesthetics - how visually pleasing the UI is
- Consistency - How consistent are the aesthetics, layouts and interactions across different parts of the UI

Like the image generation performance survey, participants of the survey will be asked to rate each of these between 1 and 5.

4.5 Application Review

The following are the results of the survey on the performance of the application created for this project. While this is not the critical aspect of the paper, it still relates to the accessibility of the models and is a good indicator of whether image generation tools can be provided in a small simple piece of software.

Learnability

This criteria was met very well. This may be due to the fact that the criteria for the functionality of the application is relatively limited. It only needs to provide access to three models and previous images. This means that meeting the criteria of learnability can be done by keeping the layout simple, so that movement around the interface does not require many steps, leading to fewer operations to learn.

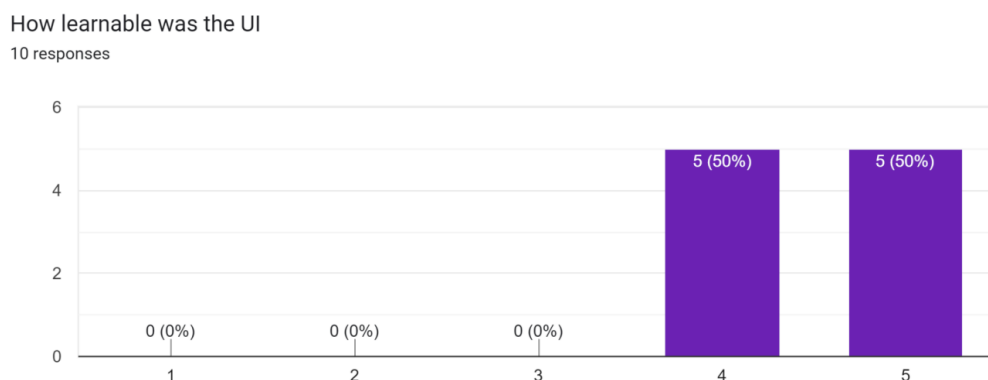


Figure 4.11: Feedback on UI Learnability

Efficiency

According to the survey, the user interface was efficient to use. Similarly to the learnability section, this is probably due to the simplicity of the application. Because there is limited functionality that needs to be implemented, keeping the user interface small, with a low number of pages is feasible. Allowing for fast movement and access between all aspects of the application.

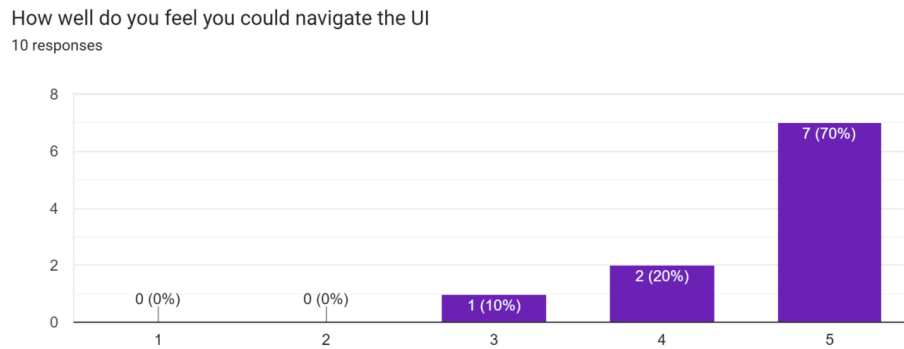


Figure 4.12: Feedback on UI Navigation Efficiency

Aesthetics

This is the part of the application that scored the lowest. While aesthetics are an inherently subjective area, in general, keeping the design simple works. However, some features such as the gradient on the background with the dark theme may have put off some users. To improve this it is likely that incorporating features such as a light mode and a simpler background may have led to better results, as the option to switch between light and dark would allow users to choose what suits them.

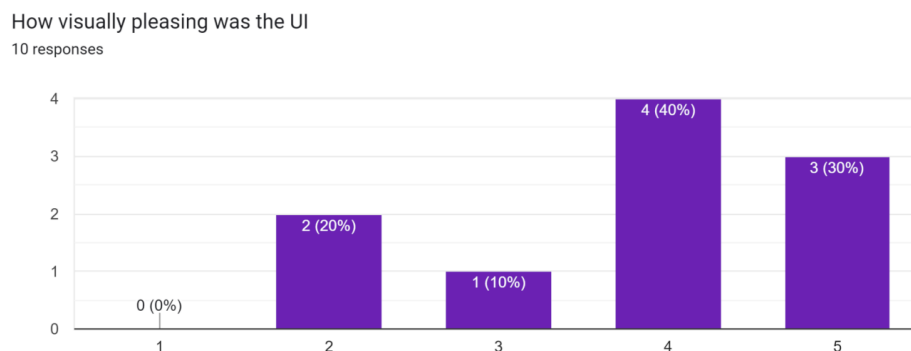


Figure 4.13: Feedback on UI Aesthetics

Consistency

Again this scored highly. Likely because there is little in the way of features because the application is so simple. This means keeping features following a design pattern is less challenging. On top of this the use of the library 'CustomTkinter' Allows for preset themes to be used such as colour themes and light or dark modes.

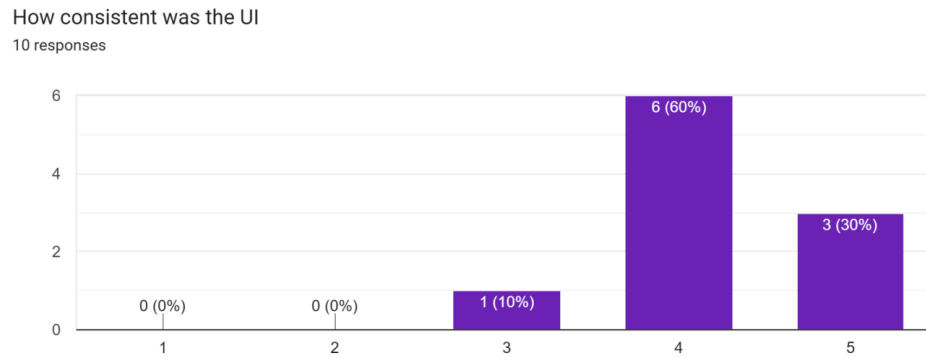


Figure 4.14: Feedback on UI Consistency

Chapter 5

Conclusion

In summary, this project aimed to explore the possibility of tackling one of the more computationally intense tasks, that being image generation on a low-power device. While this is somewhat optimistic the results show that even though the results are not on the level of large diffusion models, when using the latest techniques, reasonable results can be attained.

5.1 Achievements and Improvements

To evaluate the success and achievement of this project, I will compare the final outcomes with the initial aims of the paper listed in section 1.2, giving a brief summary of each conclusion reached during evaluation, and propose improvements that could be made.

Investigate and explore possible approaches to training image-generation models

assessing the most feasible approaches for this project's specification was important to reach a valid conclusion. This section was done well, as it gave a clear direction to head in terms of network architecture. Given more time this could be improved by going beyond purely academic review and analysis, by implementing and performing preliminary tests for each of the main architectures. Providing justification of implementation details that also pulls on empirical evidence, strengthening the results and conclusions reached in the paper.

Implement the most promising techniques and train using a large image dataset

Being met reasonably well, the images that the models were able to generate often were high quality given the size of the models. However, compared to the large models being used for comparison the performance was lacking. In part, this was down to the lack of complexity of the networks, but specifically in reference to this specification point, the dataset could have been utilized better. More specifically, performing operations, like rotations and brightness shifting for example would have allowed each network to learn more from the dataset, improving their performance relative to the size of the original dataset.

Create a clear usable Application that allows the use of the produced models

Most users rated the application and its usability highly, but the main drawback was the user interface. In particular, it produced divisive scores when users were surveyed, with some feedback suggesting it was aesthetically appealing, and other feedback saying the opposite. To improve the application, adding customizability such as light/dark themes, and layout options may allow the application to appeal to a broader range of users.

Use qualitative and quantitative methods to assess the relative performance of these models

With the use of the metrics of the Inception score and the Fréchet Inception Distance, I was able to perform quantitative evaluation and using surveys, measure performance quantitatively. In general, these two approaches agreed with each other, yielding similar results, supporting the validity of each other and indicating that the evaluation of the project was a success. To build upon this, more feedback could be gathered to improve its precision.

5.2 Future Work

Many directions could be taken if this project were to continue, as accessibility to the use and creation of neural networks spans much wider than just image generation. Although an image processing task was chosen because it is one area in which computational cost is particularly high, in almost all fields the size of models is growing rapidly in complexity. One particular area that is becoming important regarding ethical issues, including but not limited to the democratization of AI is natural language processing. With large language models (LLMs) such as ChatGPT showing themselves to be incredibly powerful tools, it is essential to investigate and demonstrate how they can be used and distributed fairly.

Another direction the research could take given the results would be to measure how quickly the performance of models approaches the state-of-the-art ones with respect to computational resources (using measures such as GPU time for example). This would further inform and build on the results of this paper, because although it is clear that home computers cannot train models in the same way the resources used by companies such as OpenAI, there is no information on how big that gap is. Having this information would allow anyone to tailor their hardware purchases to specific tasks, giving a rough idea of what performance they can expect.

5.3 Reflection and Conclusion

The aim of this project was to assess whether recent state-of-the-art techniques in image generation are able to improve the possibility of training neural networks from scratch to generate

images, and if so, demonstrate this process. Doing so by comparing the performance of these techniques implemented on lightweight models with very large models also using diffusion. In addition, to assess the performance gains of these methods, well-understood approaches such as generative adversarial networks were created to use as a benchmark.

Overall the results indicated that while diffusion techniques show a marked improvement over previous approaches, producing promising results, there is still a large gap to bridge between 'large' models. This is unsurprising due to the inherently demanding nature of the training methods based on backpropagation. This suggests that as long as training neural networks necessitates large computational resources there will always be a disconnect between the size of models the average person can train and the state-of-the-art ones. Specifically, the amount of memory available and number of operations that can be done per second directly affect how large of a network you can train in a reasonable amount of time.

Bibliography

- [1] S. Bond-Taylor, A. Leach, Y. Long, and C. G. Willcocks. Deep generative modelling: A comparative review of vaes, gans, normalizing flows, energy-based and autoregressive models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(11):7327–7347, 2022.
- [2] A. Borji. Generated faces in the wild: Quantitative comparison of stable diffusion, mid-journey and dall-e 2, 2023.
- [3] M. Cahyadi, M. Rafi, W. Shan, H. Lucky, and J. V. Moniaga. Accuracy and fidelity comparison of luna and dall-e 2 diffusion-based image generation systems. In *2023 IEEE International Conference on Industry 4.0, Artificial Intelligence, and Communications Technology (IAICT)*, pages 108–112, 2023.
- [4] H. Cai, C. Gan, J. Lin, and S. Han. Network augmentation for tiny deep learning, 2022.
- [5] T.-Y. Chang and C.-J. Lu. Tinygan: Distilling biggan for conditional image generation, 2020.
- [6] M. Cho, K. A. Vahid, Q. Fu, S. Adya, C. C. D. Mundo, M. Rastegari, D. Naik, and P. Zatloukal. edkm: An efficient and accurate train-time weight clustering for large language models. *IEEE Computer Architecture Letters*, 23(1):37–40, 2024.
- [7] J. Deng and M. Yi. Pancreas segmentation based on multi-scale convolution and attention module. In *2023 8th International Conference on Intelligent Computing and Signal Processing (ICSP)*, pages 942–946, 2023.
- [8] R. Dhiman, G. Joshi, and R. Challa. A deep learning approach for indian sign language gestures classification with different backgrounds. *Journal of Physics: Conference Series*, 1950:012020, 08 2021.
- [9] Q. Feng, C. Guo, F. Benitez-Quiroz, and A. Martinez. When do gans replicate? on the choice of dataset size, 2022.
- [10] G. Franzese, S. Rossi, L. Yang, A. Finamore, D. Rossi, M. Filippone, and P. Michiardi. How much is enough? a study on diffusion times in score-based generative models. *Entropy (Basel)*, 25(4):633, Apr 2023.

- [11] L. Gonog and Y. Zhou. A review: Generative adversarial networks. In *2019 14th IEEE Conference on Industrial Electronics and Applications (ICIEA)*, pages 505–510, 2019.
- [12] P. Guerraoui, P. Guirguis, P.-M. Kermarrec, and P. L. Merrer. Fegan: Scaling distributed gans, 2020.
- [13] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network, 2015.
- [14] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models, 2020.
- [15] B. Huang, Y. Zhu, X. Zhou, and H. Chen. Conditional inpainting generative flow. In *2022 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 1312–1318, 2022.
- [16] X. Liu, F. Xing, J. L. Prince, A. Carass, M. Stone, G. E. Fakhri, and J. Woo. Dual-cycle constrained bijective vae-gan for tagged-to-cine magnetic resonance image synthesis. In *2021 IEEE 18th International Symposium on Biomedical Imaging (ISBI)*, pages 1448–1452, 2021.
- [17] D. Mack. [online]available: <https://medium.com/octavian-ai/a-simple-explanation-of-the-inception-score-372dff6a8c7a>, 2019.
- [18] M. Padala, D. Das, and S. Gujar. Effect of input noise dimension in gans, 2020.
- [19] A. Radford, L. Metz, and S. Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks, 2016.
- [20] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen. Hierarchical text-conditional image generation with clip latents, 2022.
- [21] A. Razavi, A. van den Oord, and O. Vinyals. Generating diverse high-fidelity images with vq-vae-2, 2019.
- [22] A. Razavi, A. van den Oord, and O. Vinyals. Generating diverse high-fidelity images with vq-vae-2. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- [23] Y. Ren, X. Yu, J. Chen, T. H. Li, and G. Li. Deep image spatial transformation for person image generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.
- [24] P. Reviriego and E. Merino-Gómez. Text to image generation: Leaving no language behind, 2022.

- [25] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10674–10685, Los Alamitos, CA, USA, jun 2022. IEEE Computer Society.
- [26] C. Saharia, P. Chan, P. Chang, P. Lee, P. Ho, P. Salimans, P. Fleet, and P. Norouzi. Palette: Image-to-image diffusion models, 2022.
- [27] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen. Improved techniques for training gans, 2016.
- [28] N. Siddique, S. Paheding, C. P. Elkin, and V. Devabhaktuni. U-net and its variants for medical image segmentation: A review of theory and applications. *IEEE Access*, 9:82031–82057, 2021.
- [29] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *Proceedings of the 32nd International Conference on International Conference on Machine Learning - Volume 37*, ICML'15, page 2256–2265. JMLR.org, 2015.
- [30] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need, 2023.
- [31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin. Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [32] W. Wang, J. Bao, W. Zhou, D. Chen, D. Chen, L. Yuan, and H. Li. Semantic image synthesis via diffusion models, 2022.
- [33] A. Wijaya, Kefry, W. Wihalim, and A. A. Santoso Gunawan. The effect of ui/ux design on user satisfaction in online art gallery. In *2021 1st International Conference on Computer Science and Artificial Intelligence (ICCSAI)*, volume 1, pages 120–125, 2021.
- [34] Y. Yu, W. Zhang, and Y. Deng. Frechet inception distance (fid) for evaluating gans. *China University of Mining Technology Beijing Graduate School*, 2021.
- [35] K. Yuichi, S. Takuya, M. Masahiko, and F. Makoto. Face image generation with features of specific group from small dataset. In *2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 1119–1124, 2021.
- [36] H. Üzen, M. türkoğlu, M. Aslan, and D. Hanbay. Depth-wise squeeze and excitation block-based efficient-unet model for surface defect detection. *The Visual Computer*, 39, 03 2022.